# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Artificial Intelligence and Data Science

## ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

| | | | |
|---|---|---|---|
| **Course Code:** | DSA03 | **Course Title:** | Natural Language Processing |
| **CO Assessed:** | CO1 – Imitation (L1) | **Assessment:** | Assessment Tool 1 – Application Based Questions |
| **Weightage:** | 40% of CIA | | |
| **CO1 Statement:** | Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3) | | |
| **Student Name:** | K Sumanth Kumar Reddy | **Reg. No.:** | 192425222 |
| **Section / Batch:** | AIML | **Date:** | 23-10-2026 |

**Code of Conduct**

I K Sumanth Kumar Reddy (192425222) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K Sumanth

**Instructions**

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

| Section / Topic | Focus | Q Nos |
|---|---|---|
| **Information Extraction** | Regex-Based Information Extraction | 1 |
| **Pattern Matching** | Regex Pattern Matching | 2 |
| **Regular Expression Patterns** | Regex-Based Input Validation | 3 |

## Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

**Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:**

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

**SOLUTION CODE :**

```
“
import re
import json

# Sample resumes in text format
resumes = [
    """
    Name: Rahul Sharma
    Email: rahul.sharma@gmail.com
    Mobile: +91 9876543210
    Skills: Python, SQL, Machine Learning, NLP
    Experience: 3 years
    """,
    """
    Name: Priya Reddy
    Email: priya.reddy@yahoo.com
    Mobile: 9123456789
    Skills: Java, SQL, NLP
    Experience: 4 years
    """,
    """
    Name: Arjun Kumar
    Email: arjun.kumar@outlook.com
    Mobile: 9988776655
    Skills: Python, Java, SQL
    Experience: 1 year
    """,
    """
    Name: Sneha Patel
    Email: sneha.patel@gmail.com
    Mobile: +91-9876501234
    Skills: Python, Java, Machine Learning
    Experience: 2 years
    """
]

# Regular expression patterns
name_pattern = r"Name\s*:\s*([A-Za-z]+(?:\s+[A-Za-z]+)*)"
email_pattern = r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b"
mobile_pattern = r"(?<!\d)(?:\+91[-\s]?)?[6-9]\d{9}(?!\d)"
experience_pattern = (
    r"Experience\s*:\s*(\d+(?:\.\d+)?)\s*"
    r"(?:years?|yrs?)\b"
```

```
)

# Technical skills to detect
technical_skills = [
    "Python",
    "Java",
    "SQL",
    "Machine Learning",
    "NLP"
]

def extract_resume_info(resume):
    # Extract candidate name
    name_match = re.search(name_pattern, resume, re.IGNORECASE)
    name = name_match.group(1).strip() if name_match else "Not found"

    # Extract email
    email_match = re.search(email_pattern, resume)
    email = email_match.group(0) if email_match else "Not found"

    # Extract mobile number
    mobile_match = re.search(mobile_pattern, resume)
    mobile = mobile_match.group(0) if mobile_match else "Not found"

    # Extract technical skills
    skills_found = []
    for skill in technical_skills:
        pattern = r"(?<!\w)" + re.escape(skill) + r"(?!\w)"
        if re.search(pattern, resume, re.IGNORECASE):
            skills_found.append(skill)

    # Extract years of experience
    exp_match = re.search(
        experience_pattern, resume, re.IGNORECASE
    )
    experience = float(exp_match.group(1)) if exp_match else 0

    # Eligibility: at least 2 years and Python skill
    eligible = experience >= 2 and "Python" in skills_found

    # Return structured candidate profile
    return {
        "Name": name,
        "Email": email,
        "Mobile": mobile,
        "Technical Skills": skills_found,
        "Experience (Years)": experience,
        "Eligible": "Yes" if eligible else "No"
    }


# Process all resumes
candidate_profiles = []

for index, resume in enumerate(resumes, start=1):
    profile = extract_resume_info(resume)
    profile["Candidate ID"] = index
    candidate_profiles.append(profile)

# Display all candidate profiles
print("=" * 65)
print("       RESUME INFORMATION EXTRACTION SYSTEM")
print("=" * 65)
```

```
for profile in candidate_profiles:
    print("\nCandidate ID:", profile["Candidate ID"])
    print(json.dumps(profile, indent=4))

# Display eligible candidates
print("\n" + "=" * 65)
print("CANDIDATES WHO MEET MINIMUM ELIGIBILITY CRITERIA")
print("Criteria: Experience >= 2 years AND Python skill")
print("=" * 65)

eligible_candidates = [
    candidate for candidate in candidate_profiles
    if candidate["Eligible"] == "Yes"
]

if eligible_candidates:
    for candidate in eligible_candidates:
        print(
            f"Name: {candidate['Name']} | "
            f"Experience: {candidate['Experience (Years)']} years | "
            f"Skills: {', '.join(candidate['Technical Skills'])}"
        )
else:
    print("No eligible candidates found.")

print("\nTotal resumes processed:", len(candidate_profiles))
print("Eligible candidates:", len(eligible_candidates))
"
```

**OUTPUT:**

```
--- Candidate Profile ---
Name: Rahul Sharma
Email: rahul@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL
Experience: 3 years
Eligibility: Eligible

--- Candidate Profile ---
Name: Priya Reddy
Email: priya@gmail.com
Mobile: 9123456780
Skills: Java, SQL, NLP
Experience: 4 years
Eligibility: Not Eligible

--- Candidate Profile ---
Name: Arjun Kumar
Email: arjun@gmail.com
Mobile: 9988776655
Skills: Python, Machine Learning, NLP
Experience: 1 years
Eligibility: Not Eligible

--- Shortlisted Candidates ---
Rahul Sharma
>>
```

## Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

**Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:**

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

SOLUTION CODE :

“

```
import re

# Sample product database
products = [
    "Apple iPhone 15",
    "Apple MacBook Air",
    "Samsung Galaxy S24",
    "Samsung Galaxy Buds",
    "Sony Wireless Headphones",
    "Sony Bravia TV",
    "Dell Laptop",
    "HP Laptop",
    "OnePlus Smartphone",
    "Logitech Wireless Mouse",
    "Apple iPad",
    "Samsung Smart Watch"
]

# Function to search products using Regular Expressions
def search_products(keyword, search_type):
    matching_products = []

    # Escape special regex characters in user input
    keyword = re.escape(keyword)

    # Define pattern based on search type
    if search_type == "exact":
        pattern = r"^" + keyword + r"$"
    elif search_type == "prefix":
        pattern = r"^" + keyword
    elif search_type == "suffix":
        pattern = keyword + r"$"
    elif search_type == "partial":
        pattern = keyword
    else:
        print("Invalid search type!")
        return []

    # Case-insensitive matching
    for product in products:
        if re.search(pattern, product, re.IGNORECASE):
            matching_products.append(product)

    return matching_products


# Function to display results and generate report
def generate_report(keyword, search_type):
    results = search_products(keyword, search_type)
```

```
    print("\n" + "=" * 55)
    print("Search Type :", search_type.upper())
    print("Search Keyword:", keyword)
    print("-" * 55)

    if results:
        print("Matching Products:")
        for product in results:
            print("-", product)
    else:
        print("No matching products found.")

    print("-" * 55)
    print("Total Matching Products:", len(results))

    return {
        "Keyword": keyword,
        "Search Type": search_type,
        "Total Matches": len(results),
        "Matching Products": results
    }

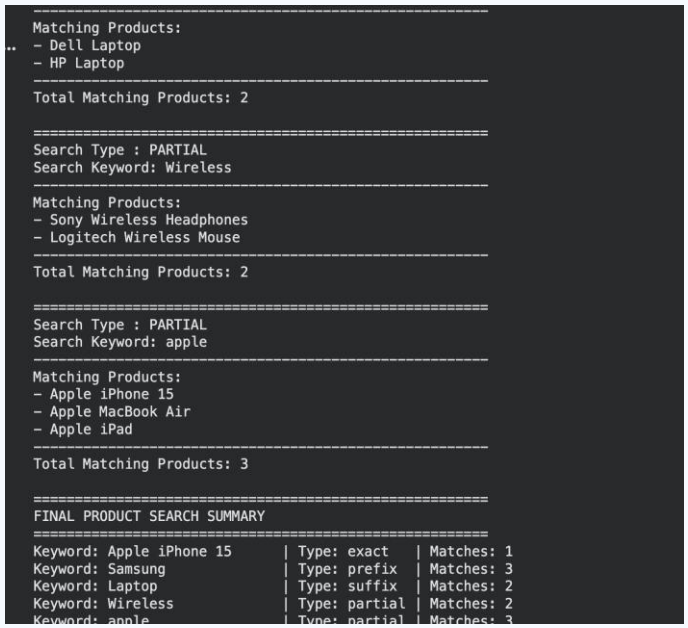

# Perform different types of searches
search_report = []

search_report.append(generate_report("Apple iPhone 15", "exact"))
search_report.append(generate_report("Samsung", "prefix"))
search_report.append(generate_report("Laptop", "suffix"))
search_report.append(generate_report("Wireless", "partial"))
search_report.append(generate_report("apple", "partial"))

# Final summary report
print("\n" + "=" * 55)
print("FINAL PRODUCT SEARCH SUMMARY")
print("=" * 55)

for report in search_report:
    print(
        f"Keyword: {report['Keyword']:<20} | "
        f"Type: {report['Search Type']:<7} | "
        f"Matches: {report['Total Matches']}"
    )
„
```

**OUTPUT :**




## Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

**Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:**

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

SOLUTION CODE :

“

```
import re

# Student registration details
student = {
    "Register Number": "192425028",
    "Institutional Email": "192425028@college.edu.in",
    "Course Code": "CSE101",
    "Semester": "Semester 3",
    "Mobile Number": "9876543210"
}

# Regular expression patterns
patterns = {
    "Register Number": r"^\d{9}$",
    "Institutional Email": r"^[A-Za-z0-9._%+-]+@college\.edu\.in$",
    "Course Code": r"^[A-Z]{3}\d{3}$",
    "Semester": r"^Semester\s[1-8]$",
    "Mobile Number": r"^[6-9]\d{9}$"
}

# Validation function
def validate_field(field, value, pattern):
    if re.fullmatch(pattern, value, re.IGNORECASE):
        print(f"{field}: {value} -> Valid")
```

```
        return True
    else:
        print(f"{field}: {value} -> Invalid")
        return False

# Validate all student details
print("=" * 55)
print("     UNIVERSITY REGISTRATION VALIDATION")
print("=" * 55)

validation_results = {}

for field, value in student.items():
    validation_results[field] = validate_field(
        field, value, patterns[field]
    )

# Generate final registration report
print("\n" + "=" * 55)
print("        FINAL REGISTRATION STATUS REPORT")
print("=" * 55)

for field, is_valid in validation_results.items():
    print(f"{field}: {'PASS' if is_valid else 'FAIL'}")

if all(validation_results.values()):
    print("\nRegistration Status: SUCCESSFUL")
    print("All student details are valid.")
else:
    print("\nRegistration Status: FAILED")
    print("Please correct the invalid student details.")

print("=" * 55)
"
```

OUTPUT :

```
=============== RESTART: C:/personal/Reinforcement Learning/3.py ===============
=======================================================
     UNIVERSITY REGISTRATION VALIDATION
=======================================================
Register Number: 192425222 -> Valid
Institutional Email: 192425222@college.edu.in -> Valid
Course Code: CSE101 -> Valid
Semester: Semester 3 -> Valid
Mobile Number: 9876543210 -> Valid

=======================================================
        FINAL REGISTRATION STATUS REPORT
=======================================================
Register Number: PASS
Institutional Email: PASS
Course Code: PASS
Semester: PASS
Mobile Number: PASS

Registration Status: SUCCESSFUL
All student details are valid.
=======================================================
>>>
```

## Rubrics for Calculation

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| **Understanding of Problem Context** | 20 | Clearly understands the problem and accurately identifies all requirements | Good understanding with minor gaps | Basic understanding; some requirements unclear | Poor understanding or misinterpretation of the problem |
| **Application of Concepts** | 30 | Accurately applies appropriate concepts to solve complex problems | Applies relevant concepts correctly with minor errors | Applies basic concepts with limited accuracy | Fails to apply appropriate concepts |
| **Problem-Solving Approach** | 20 | Logical, well-structured, and efficient approach | Mostly logical approach with minor gaps | Basic approach with some inconsistencies | Illogical or unclear approach |
| **Accuracy of Solution** | 20 | Completely correct solution with proper justification | Mostly correct with minor errors | Partially correct solution | Incorrect or incomplete solution |
| **Clarity** | 10 | Clear, well-organized, and properly explained solution | Understandable with minor clarity issues | Limited clarity and explanation | Poorly presented and difficult to understand |

| Q. No. / Criteria | Understanding of Problem Context | Application of Concepts | Problem-Solving Approach | Accuracy of Solution | Clarity | Sub. Total | Total % |
|---|---|---|---|---|---|---|---|
| 1 | | | | | | | |
| 2 | | | | | | | |
| 3 | | | | | | | |

| Faculty In-charge | Course Coordinator | Program Director |
|---|---|---|
| | | |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |